INDIAN INSTITUTE OF INFORMATION TECHNOLOGY VADODARA

CS543: Web Services and Service-Oriented Architecture • Collaboration Framework

Nivaaran: Git & GitHub Collaboration Guide

Multi-Member Push Strategy, Conflict Prevention & Exact Repository File Structure

1. Team Allocation & Assignment Responsibility Matrix

Team Member	Roll Number	Assigned Submission ZIP	Target Assignment Scope
Aarambhika	20251651001	`Nivaaran_A1_A2.zip`	Assignment 01 (Project Brief & Setup) + Assignment 02 (Architecture & Schema)
Abhishek	20251651006	`Nivaaran_A3_A4.zip`	Assignment 03 (SOAP Telecom Partner) + Assignment 04 (Complaints REST Service)
Ritik	20251651078	`Nivaaran_A5_A6.zip`	Assignment 05 (HTTP Headers & ETag) + Assignment 06 (Honest Responses & Validation) + Client

2. Exact Target Repository File Structure on GitHub

Following the multi-member push sequence below guarantees that the collaborative GitHub repository matches the **exact same folder hierarchy** as the local project:

```
nivaaran/                              # Root of your GitHub repository
├── .gitignore                         # Excludes node_modules/, dist/, .env
├── README.md                          # Master repo guide, team info, links
├── ReadMe.pdf                         # Comprehensive submission report (Assignments 01 to 06)
├── ReadMe.docx                        # Editable Word document submission report
├── brief.md                           # Institutional project brief & noun/verb model
├── GitHub_Collaboration_Guide.pdf     # This collaboration workflow guide
├── docs/                              # All assignment documentation & verification
│   ├── a2/                            # Assignment 02: Design, schemas & diagrams
│   │   ├── design.pdf                 # 9-Page Times New Roman architecture document
│   │   ├── schema.sql                 # 6-Database relational DDL SQL sketch
│   │   ├── schema.png & schema.drawio  # 6-Database Entity-Relationship Diagram
│   │   └── services.png & services.drawio # 6-Microservices Architectural Diagram
│   ├── a3/                            # Assignment 03: SOAP & XML Integration
│   │   ├── integration.pdf            # Legacy partner integration documentation
│   │   ├── partner.wsdl               # Telecom SMS provider WSDL contract
│   │   ├── soap-request.xml           # Sample XML SMS dispatch request
│   │   ├── soap-response.xml          # Sample XML carrier receipt response
│   │   └── soap-fault.xml             # Sample XML SOAP fault error
│   ├── a4/                            # Assignment 04: Rebuilding in REST
│   │   ├── openapi.yaml               # OpenAPI 3.0 specification
│   │   ├── NOTES.md                   # Resource table, escalation hard choice, fallback
│   │   └── curl_transcript.txt        # Terminal cURL execution logs
│   ├── a5/                            # Assignment 05: HTTP Methods & Headers
│   │   ├── NOTES.md                   # Method map, idempotency retry plan, ETag caching
│   │   └── curl_transcript.txt        # Terminal cURL transcript for ETag & retry
│   └── a6/                            # Assignment 06: Honest Responses & Validation
│       ├── NOTES.md                   # Status map, RFC 7807 type catalogue, 422 errors
│       └── curl_transcript.txt        # Terminal cURL transcript for Problem Details
├── services/
│   └── complaints/                    # Core Implemented Runnable Microservice
│       ├── openapi.yaml               # Complete hardened contract
│       ├── package.json & lock        # Dependencies (Express, etc.)
│       ├── src/                       # Express app, accountsClient, breaker, errors, store
│       │   └── middleware/            # auth.js, headers.js, negotiation.js, rateLimit.js
│       └── tests/                     # Automated test suite (11/11 tests passing)
│           └── complaints.test.js
└── client/                            # React Web Client (Demo SPA)
    ├── package.json & vite.config.js
    └── src/                           # React components, HTTP inspector, state
```

3. Strategy: Achieving 100% Equal Contribution (33% - 33% - 33%)

To ensure the GitHub Insights/Contributor graph displays perfectly balanced technical contributions without merge conflicts, the codebase is committed in three sequential phases:

- **Phase 1 (Aarambhika):** Pushes root documentation, project brief, and Assignment 02 design architecture.
- **Phase 2 (Abhishek):** Merges Assignment 03 SOAP contracts and initial Assignment 04 Complaints microservice.
- **Phase 3 (Ritik):** Merges Assignment 05/06 advanced HTTP headers, RFC 7807 validation, test suite, and React client.

4. Step-by-Step Multi-Member Push Instructions

Phase 1: Member 1 Push Instructions (Aarambhika - A1 & A2)

Performed on Member 1's local computer to initialize the repository:

1. **Create GitHub Repository:** Create a public repository named `nivaaran` on GitHub. Go to **Settings > Collaborators > Add people**, and invite Abhishek and Ritik.
2. **Extract Phase 1 Files:** Create an empty folder (e.g. `nivaaran-repo`) and extract `Nivaaran_A1_A2.zip` directly inside it.
3. **Execute Git Commands in PowerShell / Terminal:**

```
# 1. Configure Member 1 Git identity (MUST match your registered GitHub email)
git config user.name "Aarambhika"
git config user.email "aarambhika@gmail.com"

# 2. Initialize local repository on main branch
git init
git branch -M main

# 3. Stage and commit Assignment 1 & 2 files
git add .
git commit -m "feat(a1-a2): initialize project brief, service boundaries, and database schema"

# 4. Link to GitHub remote and push initial commit
git remote add origin https://github.com/<owner>/nivaaran.git
git push -u origin main
```

Phase 2: Member 2 Push Instructions (Abhishek - A3 & A4)

Performed on Member 2's local computer AFTER Member 1 has pushed to main:

1. **Accept Invitation & Clone:** Accept the collaborator invitation on GitHub. Clone the repository:

```
git clone https://github.com/<owner>/nivaaran.git
cd nivaaran
```

2. **Extract Phase 2 Files:** Extract `Nivaaran_A3_A4.zip` directly into the cloned `nivaaran` folder. It will seamlessly merge `docs/a3/`, `docs/a4/`, and `services/complaints/`.
3. **Commit and Push:**

```
# 1. Configure Member 2 Git identity
git config user.name "Abhishek"
git config user.email "abhishek@gmail.com"

# 2. Stage new files and commit
git add .
git commit -m "feat(a3-a4): add SOAP partner contracts and Complaints REST microservice"

# 3. Push to shared repository
git push origin main
```

Phase 3: Member 3 Push Instructions (Ritik - A5 & A6 + Client)

Performed on Member 3's local computer AFTER Member 2 has pushed to main:

1. **Clone or Pull Latest Main:**

```
git clone https://github.com/<owner>/nivaaran.git
cd nivaaran
# (Or if already cloned earlier: git pull origin main)
```

2. **Extract Phase 3 Files:** Extract `Nivaaran_A5_A6.zip` directly into the `nivaaran` folder. This updates `services/complaints/` (adding middlewares and tests), adds `docs/a5/` and `docs/a6/`, and adds the `client/` folder.
3. **Verify Automated Tests & Push:**

```
# 1. Configure Member 3 Git identity
git config user.name "Ritik"
git config user.email "ritik@gmail.com"

# 2. Run automated test suite to confirm 11/11 tests pass
cd services/complaints
npm test
cd ../..

# 3. Stage, commit, and push final enhancements
git add .
git commit -m "feat(a5-a6): implement ETag caching, rate limiting, honest validation, and client"
git push origin main
```

5. Alternative: Direct Full-Repository Push (Single Machine Setup)

If your team prefers to push the entire project directly from the current local workspace in a single operation, run the following commands from the root directory:

```
# 1. Initialize git in workspace root
git init
git branch -M main

# 2. Link your GitHub remote
git remote add origin https://github.com/<owner>/nivaaran.git

# 3. Ensure dependencies are ignored
git status   # Verify node_modules/ is NOT listed

# 4. Stage and commit entire project
git add .
git commit -m "feat: complete Nivaaran campus grievance redressal platform (A1-A6)"

# 5. Push to GitHub
git push -u origin main
```

6. Golden Rules to Prevent Merge Conflicts

Golden Rule	Common Mistake to Avoid	Mandatory Prevention Practice
Rule 1: Direct Folder Extraction	Extracting zip into an extra nested folder (e.g. `nivaaran/Nivaaran_A3_A4/`).	Extract files directly into the repository root so `docs/` and `services/` merge seamlessly.
Rule 2: Pull Before Push	Modifying and pushing without pulling latest commits from previous members.	Always run `git pull origin main` before copying new files or committing.
Rule 3: Exact Email Match	Using a generic or different email in `git config user.email`.	The email configured in Git MUST match your registered GitHub account email to receive contribution credit.
Rule 4: Never Commit node_modules	Committing thousands of dependency files inside `node_modules/`.	Ensure `.gitignore` contains `node_modules/`. Collaborators install dependencies via `npm install`.
Rule 5: Never Force Push	Running `git push -f` or `--force` when an error occurs.	Force push overwrites teammates' commits. Never use `-f` on shared collaborative branches.

7. Troubleshooting & Instant Fixes

Observed Git Error	Root Cause	Immediate Resolution Command
`error: failed to push some refs`	Remote repository contains commits you don't have locally.	`git pull --rebase origin main` then `git push origin main`
`Support for password auth was removed`	GitHub requires a Personal Access Token (PAT).	Generate token in GitHub Settings > Developer Settings > Personal access tokens, paste as password.
`Commits not showing on profile graph`	Commit email does not match registered GitHub email.	`git config user.email "your-email@..."` then `git commit --amend --reset-author`
`fatal: refusing to merge unrelated histories`	Repo was initialized on GitHub with a README before local push.	`git pull origin main --allow-unrelated-histories`

Verification Summary: Following either the 3-Phase Multi-Member Push or the Direct Push guarantees that the remote repository matches the exact architecture, documentation, and source code of the Nivaaran project.